

Locara Atlas — how I'd build the MVP

Just my own notes on how I'd approach it, before writing any code

Mandar • June 16, 2026

I wanted to write this down properly instead of just keeping it in my head, because the way I think about a build usually matters more than any single feature in it. So this isn't a spec — there's already a good one for that — it's more like me thinking out loud about how I'd actually go about it, what I'd worry about, and the calls I'd make if this landed on my desk. If it reads a bit like notes to myself, that's because that's basically what it is.

What I think this is really about

When I first read the brief, my instinct was to treat it as “build a portal that shows videos.” But the more I sat with it, the more I felt that's not the real problem. Locara already shares the data through Drive folders and spreadsheets, and honestly, that works fine on a technical level. The thing that doesn't work is how it *feels* on the other end. A shared folder quietly tells the client they're buying files from a vendor. A proper, private, branded portal tells them they're working with a company they can actually build on. It's the same data either way — but it's a completely different impression, and that impression is what ends up affecting pricing and whether they stick around.

So that's the lens I'd keep coming back to. The app's real job is to make a serious robotics or AI team open it and immediately feel like the data, and the people behind it, can be trusted. And I think that trust mostly comes from two unglamorous things: the data being genuinely well presented, and nothing ever feeling broken or half-finished. I'd happily trade a few extra features for those two.

The other thing I keep reminding myself is that this is a one-to-two week build, not a platform. The scope in the brief is generous — maybe a little too generous — and the trap I'd want to avoid is trying to do all of it at a so-so level. I'd much rather get the core path working really well and make the handful of screens a client actually lives in feel good, than spread myself thin and half-finish everything. I've been on the wrong side of that trade before and it always shows.

How I'd approach it

There are a few things I'd decide right at the start, mostly so I'm not re-litigating them with myself halfway through the week.

The first, and the one I care about most, is that the data isolation has to live in the database

— not in my UI code, and not in my API handlers. A client from one company should never be able to see another company’s data, even if they start fiddling with URLs or poking around in the network tab. And honestly, I don’t fully trust myself to remember the right filter on every single query when I’m moving fast, so I’d rather let Supabase row-level security enforce it at the bottom. That way, even if I slip up somewhere in the app, the database just returns nothing for the wrong company. To me this is the most important part of the whole thing, so it’s the first thing I’d get right, not the last.

Second, the videos never touch my app. The database only stores the URL, and the browser streams the file straight from storage. That’s what keeps the app fast no matter how big the files get or how many of them there are. I’d treat the app as a thin metadata and access layer and let storage carry the weight. For the private files I’d hand out short-lived signed URLs, so a link someone copies doesn’t just stay open forever.

Third — and this is more of a personal habit than a rule — I’d build the boring spine before I make anything look nice. Login, a dashboard with real numbers, the explorer, a real video actually playing, all working end to end, even if it looks plain and a bit ugly. Once I know the data and the auth are solid, then I let myself polish. I’ve made the mistake before of making a screen beautiful and then having to tear out the data layer underneath it, and it stings every time, so I try not to anymore.

Fourth, I’d use the real dataset from day one and never reach for placeholder text. The twelve real records, seeded early. Fake data has a way of hiding the awkward cases — a title that’s a bit too long, a one-minute clip sitting next to a thirty-minute one — and I’d much rather bump into those early, while they’re cheap to fix, than discover them the night before a demo.

The order I’d build in

I’m a big believer in doing the scary things first, while there’s still time to recover if one of them turns out to be wrong.

So I’d start with the foundation: the Supabase tables, row-level security switched on and actually *tested* with two accounts in two different companies, and the real videos seeded. Then auth — login, sending admins and clients where they each belong, protecting the routes, keeping the session alive. And I genuinely wouldn’t let myself move on until I’d logged in as one client and tried, properly tried, to pull the other client’s data — and failed. If that little test doesn’t pass, nothing else I build really matters, so it’s my gate out of this stage.

Then the spine. One plain path working all the way through: log in, see a dashboard with real numbers, browse the collections, open the explorer, click a video, watch it play from storage. Ugly is completely fine here — I’m not trying to impress anyone yet. I just want to prove the whole thing holds together and that a real video actually streams the way I

expect.

After that, the screens that matter most, roughly in the order of how much time a client spends in them. The dataset explorer first, because that's where they'll really live, sizing up whether the data is any good — so the search, the filters, the sorting, the grid and list views all need to feel quick and effortless. Then the video detail page, because that's the moment someone decides whether a clip is worth it, which means the player has to handle loading and buffering and the occasional broken URL gracefully, and the metadata next to it has to read cleanly. Then the dashboard, since it's the first thing they see after login and it should quietly say “these people are serious” without shouting it. Collections after that.

Polish and the admin side I'd save for last, on purpose. The admin panel is internal — it just needs to be functional, not pretty, so it can wait. And the final stretch is honestly the stuff that separates a real product from a demo: a proper loading state for everything, friendly messages when something fails, empty states that don't look broken, and making sure it all holds up on a tablet and across the main browsers with a clean console. I'd finish by walking the whole flow myself, pretending I'm the client seeing it for the first time, and fixing anything that feels even slightly off.

The logic behind this order is pretty simple: if I'm going to run out of time — and on a tight build there's always a chance — I want to run out during the polish, not during the parts that are genuinely hard to get right. Front-loading the scary stuff means the unknowns are behind me while I still have room to react to them.

The parts I know are tricky

The explorer is the one I'd keep the closest eye on. There are only twelve videos today, but it's pretty clearly built to grow, and nine combinable filters plus live search can turn sluggish quickly if I'm lazy about it. So I'd debounce the search, build the filter options from the data rather than hard-coding them, and at this scale just filter in the browser so it feels instant — but I'd write it in a way that can move to proper database queries later without me having to redo the interface.

The video player is the other one I'd sweat a little. A clip that buffers forever, or dies into a black box, undoes all the careful work in about two seconds. So it gets a real loading state, a buffering indicator, and a clear, human message if a URL goes bad. And no autoplay — let the person press play when they're actually ready to watch.

And the data isolation, one more time, because it's the thing I'd genuinely lose a little sleep over: I don't consider it done until I've tried to break it from a second account and couldn't. Saying it's on isn't the same as knowing it works.

What I'd leave out, on purpose

Being honest about scope is most of what makes a short build succeed, so I'd rather be upfront about what I'm *not* doing this time around: no AI or semantic search, no annotation overlays or pose tracking, no bulk CSV upload, no watermarking, no per-client branding, no payments, no client-facing API. Every one of those is reasonable down the line, and a couple of them are genuinely interesting — but none of them are what makes this first version land, and chasing them is exactly how the polish week quietly disappears. The one bit of future-proofing I would do now is fill in the normalised tags, because that keeps the filtering consistent and turns search into a clean addition later instead of a painful migration.

When I'd call it done

For me, “done” was never really about the features existing. It's about whether I'd be comfortable handing a client a login and just watching them use it without hovering nervously. In practice that means: two accounts in two companies prove the data stays separate, and I couldn't break it even when I tried; a client logs in and quickly sees real numbers, real collections, and can search and filter their way to a video that plays smoothly; everything has a loading state and every failure has a message that doesn't feel like an error code; the admin can run the whole thing without ever touching the database; and the production build is clean and holds up on desktop and tablet across the main browsers.

The real test I'd use on myself is whether I can run the full demo without flinching: log in as a client, watch the dashboard come up, browse the collections, filter the explorer down to something specific, open a video and have it play quickly with clean metadata sitting beside it, submit a data request — and then log in as a completely different client and show that none of the first one's data even exists for them. If that runs clean, start to finish, then I think the MVP has done its job.

And if I had a little more time after that, I honestly wouldn't reach for new features. I'd go back and deepen the explorer, add proper access logs for the security conversation that enterprise clients always end up having, and start laying the groundwork for search on the normalised tags. Basically, more of what makes the data feel valuable, and none of what just adds surface area to maintain.

If I had to put the whole thing in one line: lock down the data, build the boring spine first, make the few screens they actually live in genuinely good — and then have the discipline to stop there.